



INTRODUCTION



The year is 2010. Global threat researchers have discovered a new piece of malware that uses several techniques to infect particular victims. It specifically targets a type of Siemens programmable logic controller used in the uranium enrichment process at Iranian nuclear facilities. The malware, eventually dubbed “Stuxnet,” profiles the environment it’s running in to ensure it infects only its intended victim. Stuxnet remained undetected for months due not only to its targeted nature but also to the multiple defense evasion and stealth techniques in its arsenal. One such technique is its use of stolen code-signing certificates, a relatively new tactic that gives the malware an air of authenticity.

Fast-forward to January 2019. A compromised ASUS software update server is serving malicious, fake updates that check the MAC address of the affected computer. The attack is especially targeted in nature: only if the victim has one of a handful of hardcoded addresses does the executable download an additional payload from the internet. Once installed, the malware remains dormant and undetected until specific triggers occur. The actors behind this attack sign their malicious files with legitimate ASUS certificates, helping the payload slip past anti-malware defenses. Researchers dub this attack “ShadowHammer.”

A short while later, in 2020, the world is rocked by a threat group that infiltrates the network of SolarWinds, a company that supplies hundreds of thousands of organizations with network- and system-moni-

toring software. The threat actors inject malicious code into a legitimate software update service for the SolarWinds Orion platform. These updates are pushed to organizations that use Orion, and the malicious code silently and effectively delivers a remote access trojan to the victim organizations' networks. The actors behind this attack remain undetected for a long time due to the techniques they employ to blend into their target environments. This attack is later nicknamed "Sunburst."

In these attacks, the world has witnessed *evasion*: threats seeking to remain hidden and undetected for as long as possible while protecting themselves from host and network defense software and investigators. Once detected, evasive threats may alter their behavior, dynamically modify their code, or terminate themselves, while destroying any evidence that they ever resided in the victim's network.

Evasive and context-aware malware is a highly effective and persistent threat that requires defenders to consistently adapt. As a result, it's vital for cybersecurity professionals and researchers to have a deep understanding of the various evasion techniques malware uses, how to identify them, and how to overcome them. This book explores the nature of evasive and context-aware malware common to Windows, providing insights into the techniques it uses to evade detection as it exploits the operating system's features and architecture. This book also aims to equip malware analysts, forensics investigators, frontline defenders, detection engineers, researchers, and students with the knowledge and tools they need to understand these types of threats and peel back the layers of armor concealing the malware's code.

Before we begin, let's define exactly what malware is by looking at some common types and how they take advantage of evasion techniques.

What Is Malware?

Generally speaking, *malware* is any piece of software that does malicious things. Seems simple enough, right? However, there are often conflicting definitions of malware. Remcos is software sold on the open internet and described by its authors as a “remote administration tool.” However, since anyone can buy Remcos (completely legally, I might add), it’s largely used for nefarious purposes and has many of the same capabilities as a known type of malware called a remote access trojan. Another example is AsyncRAT, an open source “remote access tool” that, according to its author, is “designed to remotely monitor and control other computers through a secure, encrypted connection.” So, are Remcos and AsyncRAT malware, then? The answer largely depends on whom you ask, who’s using it, and for what purpose. Context is key.

Malware can be separated into different types, or *classes*. Malware classes are defined by groupings of behaviors and capabilities in differing malware families. Some of the most common malware classes are as follows:

Remote access trojans (RATs)

RATs are used to provide a persistent connection or access to an infected system. RATs often can monitor the infected host using techniques such as logging keystrokes, issuing commands to the infected host, or downloading additional malware onto the host.

Infostealers

Infostealers typically target sensitive information on the victim host, such as login credentials, banking information, cryptocurrency wallets, cookie and browser history, and similar information. They then send that data back to the threat actor(s). Banking trojans, which

specifically target banking and financial-related data, could be considered a type of infostealer.

Droppers and loaders

Droppers and loaders are designed to deploy additional malware onto the system. Technically speaking, droppers contain an embedded payload and drop it to the victim system upon execution. Loaders, on the other hand, download their payloads from an external resource, such as the internet. However, these terms are often used interchangeably. These malware variants pave the way for the additional malware, sometimes even preparing the victim host by disabling anti-malware software and other endpoint defenses before the payload is deployed.

Ransomware

Ransomware is designed to deny a victim access to a system or data until a sum of money, often in the form of cryptocurrency, is paid to the threat actor. The malware may encrypt the hard disk or specific files on the system, “lock” access to files or programs, or otherwise prevent the victim from using their system as intended. The attacker then demands a ransom payment from the victim in exchange for restoring the systems and data.

Wipers

Wipers (or *killware*) are close cousins to ransomware. They are designed to destroy files on the victim’s system to cause damage or impact services. To accomplish this, the wipers encrypt data on the machine or erase the data using partition tools, for example. So, wipers generally act like ransomware, except that they are not intended to decrypt data after encrypting it.

Worms

Worms are a self-spreading type of malware. Once they infect a host, they often scan the victim's network, searching for additional systems to infect.

Viruses

The word *virus* is often used synonymously with *malware*, but this isn't exactly accurate: all viruses are malware, but not all malware are viruses. Viruses append malicious code to files on a victim system, and when those files are sent to another victim and opened, the virus spreads to the new victim host.

Rootkits and bootkits

Rootkits are specialized variants of malware that are designed to conceal their presence from both system users and security tools. To avoid detection, rootkits typically modify system components at the kernel level of the operating system, which allows the attacker to maintain access to the compromised system. *Bootkits* usually have the same purposes as rootkits but instead infect the master boot record (MBR) or other components of a computer system's boot process, allowing them to gain control of the system before the operating system loads.

Trojan horses

Historically, *trojans* have been defined as malicious software that masquerades as legitimate software. I include them in this list for the sake of completeness, but I don't like this term or its definition. After all, what malicious software *doesn't* masquerade as legitimate software? If malicious software told us it was malicious, we wouldn't be tricked into executing it. Hence, *trojan* is an archaic and very often overused term.

This list covers a large percentage of malware, but it's not exhaustive. Other variants of malware include keyloggers, coinminers, spyware, hacktools, and more. It's important to keep in mind that these mal-

ware types are not always straightforward, and there are often overlaps. It can be helpful to think of these as behavioral characteristics rather than distinct categories.

We often classify malware into two overarching groups: commodity and bespoke. *Commodity* malware is often available to a large market, whether on the open internet or on dark-web forums. This type of malware is used by many different threat groups at once. Examples of commodity malware include Lokibot and Agent Tesla, two of the most popular pieces of malware in circulation. *Bespoke* malware is more customized, often targeting a certain industry or even a particular company or person, and it has a very specific objective. Examples of this type of malware include Stuxnet, which we mentioned earlier, and HermeticWiper, which targeted systems in Ukraine at the beginning of the Russian invasion in 2022. Commodity malware can be augmented to be more targeted and bespoke in nature.

NOTE

Malware exists for all major operating systems, including (but not limited to) Windows, macOS, and flavors of Unix, as well as mobile operating systems such as Android and iOS. As it is the most prevalent, I've decided to focus on Windows malware in this book. However, many of the evasion techniques we'll discuss in this book can also be implemented in some form on other operating systems.

What Is Malware Analysis?

Malware analysis is the process of investigating and taking apart malicious code and software. A malware analyst's goal is to identify and understand the behavior, functionalities, and potential impact of a malware sample and the attack surrounding it (also known as its *context*). Malware analysis is as much an art as a science because it often takes a great deal of creativity to fully understand a malware sample, especially if it is one of the more advanced variants, and to put that

knowledge to use to detect and prevent future attacks. As we'll discuss in **Chapter 3**, malware analysis can be divided into two overarching methods: static analysis and dynamic analysis.

Why Does Malware Use Evasion?

The ultimate goal of evasion is self-preservation by avoiding detection and analysis. Some malware is designed to remain embedded in a victim's system or network for as long as possible. Other malware simply seeks to circumvent as many network and host defenses as it can before being detected so that it can quickly execute its payload. Malware authors may implement evasion techniques in their malware for any of the following reasons:

Hampering analysis

Intelligent malware knows that at some point, it will be detected and likely investigated by an analyst or researcher in a virtual machine or malware sandbox. It's becoming increasingly common to witness malware scanning its host system and looking for indications that it's being run in an analyst's lab. The malware may also search for signs of analysis tools, such as code debuggers, and interfere with them to prevent, thwart, or at least slow down malware analysts' efforts to understand its underlying behaviors, functionalities, and code.

Evading defenses

Network and host defenses such as an intrusion prevention system (IPS), anti-malware, and endpoint detection and response (EDR) products are nuisances to malware. Evasive threats will try to circumvent and bypass these defenses to remain hidden on the infected host.

Targeting systems and profiling context

Threats such as Stuxnet go to great lengths to identify the type of system they're currently running on. Malware that implements profiling techniques may attempt to determine the victim's operating system, the software installed on the victim's machine, or even the victim's physical location. The malware then uses this information to determine whether the system is a valid target or not. If not, the malware may remove all evidence that it ever resided on the victim host, thereby eluding detection. Malware may also use profiling to determine the specific defenses employed in the target system or network and alter its behaviors and capabilities based on that information.

Why I Wrote This Book

Over my years of dedicated study and research into the realm of cybercrime and its perpetrators, I've seen an increase in the use of evasion techniques, even in the most rudimentary and widespread malware. Modern malware combines multiple tactics to circumvent the most robust sandboxes and defenses and to impede analysis and investigation as much as possible. Techniques that once were reserved for more advanced or bespoke malware are becoming much more common. Not only that, but evasive measures in malware are constantly evolving to further thwart analysis efforts.

This book is intended to serve as both an introduction to and an extensive resource on Windows-based malware evasion techniques. It can be challenging for newcomers to this field and seasoned professionals alike to identify and learn the fundamentals to combat evasive threats. While substantial research is being conducted in this domain, it still remains relatively niche. I firmly believe that the better we malware researchers and security analysts understand modern threat behavior and emerging trends, the more effectively we can defend our organizations and protect future victims.

My hope is that after reading this book, you will have clear strategies that you can readily incorporate into your malware analysis method-

ologies or your organization's defensive measures. Above all, my goal is to spark your interest in seeking further knowledge in this field. Together, we must keep threat actors and malware authors on their toes.

While I consider myself knowledgeable in this subject area, I am well aware that there is always room for growth and learning. Please do not hesitate to reach out to me personally with any questions, feedback, or additional insights into the material presented in this book. I always enjoy engaging in thoughtful discussion about cyberthreats and malware.

Who Should Read This Book

I wrote this book for anyone who seeks to better understand evasion techniques used by modern and advanced malware. Perhaps you're already a malware researcher who wants to explore how malware can evade and circumvent your analysis tools and analysis lab environment. Maybe you're a frontline incident responder seeking to better understand how to identify and detect these types of threats, or perhaps you're a forensics analyst trying to determine how to investigate systems compromised with advanced malware. This book is for you.

This book is very technical in nature and is not a beginner's guide to Windows malware analysis, so I assume you have at least an intermediate-level knowledge of cybersecurity principles and a basic understanding of malware analysis concepts. Ideally, you'll also have experience reversing assembly code. If you're new to these topics, however, have no fear: the first three chapters of this book provide a crash course in malware analysis and the fundamental concepts required to understand the later chapters of the book.

Also, I expect that you have a malware analysis lab environment set up to safely execute malware. This is very important, as all examples

in this book use real malware samples. **Appendix A** includes a guide for setting up a hypervisor and virtual machines for safe malware analysis.

How This Book Is Organized

This book is organized into 4 parts, which are made up of 17 chapters, plus 3 appendixes.

Part I, The Fundamentals, establishes a baseline level of knowledge for the rest of the book.

Chapter 1: Windows Foundational Concepts Covers fundamental concepts of the Windows operating system

Chapter 2: Malware Triage and Behavioral Analysis Focuses on the basics of triaging malware and analyzing the behavior of malware samples to determine how they operate on an infected system

Chapter 3: Static and Dynamic Code Analysis Covers the fundamentals of static and dynamic code analysis and how you can use these techniques to uncover malware's true intentions

Part II, Context Awareness and Sandbox Evasion, delves into how evasive malware is able to detect virtual machines and malware analysis sandboxes.

Chapter 4: Enumerating Operating System Artifacts Discusses how malware can closely inspect underlying operating system artifacts to detect analysis efforts

Chapter 5: User Environment and Interaction Detection Explains the ways in which malware thwarts investigations by enumerating user interactions and the environment in which it's running

Chapter 6: Enumerating Hardware and Network Configurations

Examines how malware can inspect system hardware and network settings to discover a malware analyst's sandboxes and virtual machines

Chapter 7: Runtime Environment and Virtual Processor

Anomalies Covers how processing and runtime environment anomalies can tip off malware to analysis attempts

Chapter 8: Evading Sandboxes and Disrupting Analysis Explores several other techniques a threat actor can use to completely evade and disrupt an analysis environment

Part III, Anti-reversing, details the methods attackers use to complicate the reverse engineering process for malware analysts.

Chapter 9: Anti-disassembly Explains how malware uses anti-disassembly techniques to prevent and disrupt manual code analysis

Chapter 10: Anti-debugging Discusses how malware is able to detect and circumvent debuggers and dynamic code analysis

Chapter 11: Covert Code Execution and Misdirection

Demonstrates ways in which malware can execute code covertly or confuse and misdirect malware analysts

Part IV, Defense Evasion, delves into how malware evades defensive controls.

Chapter 12: Process Injection, Manipulation, and Hooking

Reveals how malware can inject malicious code into different processes, manipulate processes, and hook function code

Chapter 13: Evading Endpoint and Network Defenses Covers how malware evades and circumvents network and endpoint defenses

Chapter 14: Introduction to Rootkits Discusses the fundamentals of an especially dangerous type of evasive malware: rootkits

Chapter 15: Fileless, Living Off The Land, and Anti-forensics

Techniques Explores how malware uses so-called fileless techniques and anti-forensics measures to evade defenses and forensics tools

Chapter 16: Encoding and Encryption Focuses on encoding and encryption techniques, providing practical methodologies for analyzing malware

Chapter 17: Packers and Unpacking Malware Discusses how malware obfuscators and packers work, going into depth on how to unpack malicious code

The appendixes include a walk-through of building a malware analysis lab; a list of Windows API functions that malware can exploit for evasion purposes; and references for further reading on the world of malware and malware analysis.

Malware Samples for This Book

Throughout this book, I include analysis labs and information about specific malware samples and families. I often reference the malware file's signature in the format `SHA256:hash_value`. Here's an example:

```
SHA256:b625df3af182060e3ee589f95669a76f43840f93df0a66fb942af51895de5504
```

Most of the malware samples I reference in this way can be downloaded from VirusTotal (<https://www.virustotal.com>), assuming you have a commercial account, or are available for free from MalShare (<https://malshare.com>). Please note that these malware samples are *real* malware. Before downloading and executing any of the malware I reference in this book, make sure that you've built a dedicated mal-

ware analysis lab and configured it for safe analysis. **Appendix A** discusses tips on how to do this.

Finally, for this book, I've tried to use a mix of malware samples, including both 32-bit and 64-bit malware. You may be wondering why I focus so much on 32-bit code when 64-bit code is largely replacing it. The simple answer is that 32-bit malware is still common, likely because there's no motivation for malware to move to a 64-bit architecture. It doesn't need the extra memory address space or performance that 64-bit architectures supply. And perhaps most importantly, 32-bit malware will run on nearly every version of Windows. Keep in mind that some people are still running outdated operating systems such as Windows XP, Windows Server 2003, and Windows 7, as well as older processor architectures.

Now, let's start digging into the foundational concepts of Windows. See you in **Chapter 1**.